1. Executive summary

Sanrakshan is a human-in-the-loop disaster decision-support platform for flood and fire incidents. It transforms noisy satellite, weather, camera, sensor, citizen-report, and helpline signals into a spatial confidence estimate. The platform deliberately asks people for confirmation only when uncertainty is material, then requires an accountable operator to approve, adjust, or reject every response recommendation.

The product is not an autonomous emergency-dispatch system. It supports trained decision-makers with traceable evidence, calibrated confidence, and a simulated responder-coordination workflow suitable for academic evaluation.

- Primary outcome: fewer false dispatches without delaying verified incidents.
- Core evaluation measures: false-dispatch rate, confirmation latency, verifier cost, calibration error, and operator traceability.

2. Product definition, scope and users

The initial release supports flood-water and visible fire/smoke hazards. Each incident is a geospatial case with source observations, model outputs, human evidence, confidence history, recommendations, and immutable decisions. The system is designed for a smart-city emergency-control-room setting.

- Administrator: configures users, policies, sources, thresholds, and retention.
- Operator: reviews explanations and approves, adjusts, or overrides recommendations.
- Verifier: receives nearby tasks and supplies geotagged observations/evidence.
- Responder viewer: sees approved simulated assignments and status only.

In scope	Out of scope
Flood/fire detection, confidence fusion, verification tasking, explanation, approval, simulated dispatch	Direct control of police/fire/ambulance systems; autonomous emergency actions

Responsive PWAs, source adapters, routing simulation, auditability	Native mobile app, production public-warning broadcasts, medical advice

3. Functional and quality requirements

The product must preserve the origin, time, geometry, and reliability of every signal; show why a confidence changed; and prevent an unapproved recommendation from becoming a dispatch. A late or duplicate message must not create a second incident or task.

- FR-01 ingest and normalise source observations.
- FR-02 cluster observations into an incident and maintain a confidence timeline.
- FR-03 create, rank, time out, and escalate verifier tasks only in the ambiguity band.
- FR-04 fuse verifier evidence and generate a human-readable recommendation.
- FR-05 record approve/adjust/override actions with actor, reason, and before/after values.
- NFR: 99.5% staging-service availability target; P95 read API under 500 ms; auditable decisions retained per policy; graceful degradation when an adapter fails.

Logical architecture

The following layered view is deliberately service-oriented: it isolates real-time ingestion, ML inference, human decision-making, and durable evidence so that a detection can never directly command a field response.

Experience layer	Decision & orchestration	Data & intelligence
Operator PWA: map, queue, explanation, approval/override	API Gateway and Incident Orchestrator	PostgreSQL + PostGIS: incidents, geometry, RBAC, audit
Verifier PWA: task, location, photo/video, response	Kafka event backbone; Notification, Dispatch and TMVN services	Neo4j: verifier/network relationships; S3: encrypted evidence

Responder viewer: approved assignments only	Python ML services: fire, flood, NLP, Bayesian fusion	Model registry, feature snapshots, telemetry and logs

Data flow: external sources and user evidence enter through adapters; normalised events are published to Kafka; perception and fusion services update an incident confidence; ambiguous incidents create verifier tasks; only an operator-approved recommendation produces a simulated dispatch event.

4. End-to-end incident lifecycle

1. An adapter validates and normalises a weather, FIRMS, CCTV, IoT, helpline, or citizen observation. 2. The Incident Orchestrator deduplicates it by spatial-temporal proximity and publishes it. 3. Perception and NLP agents score the signal. 4. Bayesian fusion calculates posterior risk and uncertainty. 5. High certainty routes to operator review; low risk is monitored; ambiguous confidence opens TMVN verification. 6. Evidence is checked, fused, and explained. 7. The operator records an approval, adjustment, or override. 8. An approved action creates a simulated dispatch and complete audit trail.

5. Frontend and user experience

Both React applications are responsive PWAs. The Operator Command Dashboard is optimized for a desktop control room; the Verifier Portal is optimized for a mobile browser and deliberately remains usable over slow networks. Leaflet or Mapbox GL renders incidents, evidence, risk cells, routes, and responder coverage.

- Operator screens: sign-in, live map, incident queue, incident evidence timeline, explanation panel, task tracker, approve/adjust/override modal, audit search, policy administration.

- Verifier screens: task invitation, consent/safety notice, map and route, evidence capture/upload, observation form, submit/decline, history and reputation summary.

- Accessibility: keyboard navigation, textual map alternatives, high contrast, semantic labels, error summaries, and no colour-only status encoding.

6. API, services and event contracts

Spring Boot exposes versioned /api/v1 endpoints behind the gateway. Each mutating request accepts an Idempotency-Key and returns a correlation ID. Services use an outbox pattern to publish durable Kafka

events after database commit; consumers deduplicate by event ID and send irrecoverable messages to a dead-letter topic.

Interface	Purpose	Key fields
POST /incidents	Create/merge normalised incident	hazardType, geometry, observedAt, source, correlationId
GET /incidents/{id}/explanation	Explain posterior and contributors	posterior, credibleInterval, topSignals, modelVersion
POST /verification-tasks/{id}/evidence	Submit verifier evidence	taskId, location, mediaRefs, observation, capturedAt
POST /decisions	Operator approval/adjustment/override	incidentId, action, reason, dispatchPlan
Kafka incident.updated	Publish confidence/incident changes	eventId, version, incidentId, state, occurredAt
Kafka verification.tasked	Create task notification	taskId, verifierId, expiryAt, score, incidentId

7. ML and agent design

The ML design separates perception from decision confidence. Every deployed model is versioned, calibrated, monitored, and rollback-capable. The decision service never treats raw detector probability as a dispatch command.

Agent	Primary model	Comparison/baseline	Decision use
Fire Perception	YOLOv8-S fire/smoke detector	RT-DETR-R50	Produces calibrated flame/smoke evidence, mAP and latency monitored
Flood Perception	U-Net with ResNet-34 encoder	DeepLabV3+ with ResNet-50	Produces flood-water mask, IoU/Dice monitored
NLP Triage	Fine-tuned DistilBERT	TF-IDF + Logistic Regression	Classifies text category/severity and extracts key fields
Confidence Fusion	PyMC Bayesian model	Calibrated XGBoost	Outputs posterior probability and credible interval
TMVN Selection	Explainable weighted scoring	Rule sensitivity analysis	Ranks proximity, reliability, availability, cost, severity

Fusion features include calibrated vision confidence, weather intensity, satellite anomaly, IoT level, report density, location context, source reliability, verifier reputation, and recency. Training splits must be grouped by disaster event/location to prevent near-duplicate or same-event leakage. The primary model is accepted only if it is calibrated, improves the evaluation outcomes against XGBoost, and does not increase unsafe false negatives. Drift triggers include confidence-distribution shift, degraded calibration, source missingness, and performance decline on reviewed evidence.

8. Dataset and external-source catalogue

All datasets are candidate sources. Before ingestion the team must verify current license, attribution, access terms, geographic suitability, annotation quality, and institutional approval. The training registry records source version, checksum, license, transformation, split, and exclusion reasons.

Dataset/source	Use and split strategy	Direct link
Sen1Floods11	Primary satellite flood segmentation. Split by event/country, never random tiles.	https://github.com/cloudtostreet/Sen1Floods11/blob/master/README.md
FloodNet	Fine-tune aerial post-flood scene understanding; retain official train/validation/test separation.	https://github.com/BinaLab/FloodNet-Supervised_v1.0
xBD / xView2	Optional damage context only; split by disaster event.	https://xview2.org/
Flameye Wildfire Detection	Fire/smoke object detection; retain source split and CC-BY-4.0 attribution.	https://huggingface.co/datasets/Hajorda/flameye-wildfire-detection
FireViewer corpus	Additional fire/smoke training after source-license review.	https://huggingface.co/datasets/fireviewer/fire-smoke-detection-corpus-v1
NASA FIRMS	Near-real-time thermal anomaly signal, not ground truth.	https://firms.modaps.eosdis.nasa.gov/active_fire/
Open-Meteo	Academic weather adapter; replace with approved operational provider before production.	https://open-meteo.com/en/docs

9. Database, graph and evidence storage

PostgreSQL with PostGIS is the operational system of record. Core tables are users, roles, incidents, observations, confidence_snapshots, verification_tasks, evidence, decisions, dispatches, source_health, model_versions, and audit_events. Geometry is stored in WGS84 with GiST spatial indexes; time-series queries use incident/time indexes. Neo4j records verifier-to-area, verifier-to-task, responder-to-capability, and incident-to-dependency relationships used by routing and explainability.

- S3 layout: evidence/{incidentId}/{taskId}/{sha256}; original objects encrypted, immutable after acceptance, with thumbnail/derived artifacts separately labelled.

- Retention: operational data and evidence follow approved institutional policy; audit decisions are retained longer; deletion is logged and does not erase decision metadata.

- Backups: encrypted daily RDS backups, point-in-time recovery, tested restore runbook, S3 versioning, cross-region recovery only after governance approval.

10. Security, identity and privacy

OAuth2/OIDC issues short-lived JWT access tokens and rotating refresh tokens. RBAC is checked both at gateway and service boundaries. Verifiers see only their assigned task and a deliberately limited incident context. Evidence URLs are short-lived presigned URLs; all access is audited.

- TLS 1.2+ in transit; KMS-backed encryption at rest; Secrets Manager for credentials; no secrets in code or CI logs.

- Validate MIME type, size, checksum and content; malware scan uploads; strip unsafe metadata; quarantine failed uploads.

- Apply OWASP controls: server-side validation, parameterised queries, CSP, CSRF protection where applicable, security headers, dependency scanning and least-privilege IAM.

- Threat model covers malicious uploads, credential theft, falsified evidence, replayed events, data leakage, model poisoning, denial of service, and unsafe operator automation.

11. Hosting, CI/CD, observability and scaling

AWS uses a VPC with public load balancer and private ECS/Fargate services. RDS/PostGIS, managed Kafka, Neo4j hosting, S3, Secrets Manager and CloudWatch are separated by development, staging and production accounts/environments. Docker Compose mirrors essential services locally.

- GitHub flow: protected main branch, feature branches, pull requests, review, tests, SAST/dependency/container scans, signed image publication, staging smoke test, manual production approval and tagged releases.

- Rate limits: per-IP login protection; role-aware API limits; upload quotas; Kafka consumer back-pressure; 429 responses include Retry-After.

- Logs are structured JSON with correlationId, incidentId and eventId; errors route to Sentry-equivalent tracking; dashboards show queue lag, API latency, source freshness, task expiry, model confidence and costs.

- Alerts: error-budget burn, P95 latency, dead-letter growth, source silence, failed backups, high override rate, calibration drift and anomalous upload rejection.

- Scale stateless APIs and consumers horizontally; partition Kafka by incident/geography; use read replicas and caching for map reads; degrade to monitor-only when fusion inputs are insufficient.

12. Test and acceptance strategy

Tests span unit, integration, API contract, event replay, end-to-end PWA, accessibility, security, performance, chaos/failure, ML, usability and acceptance testing. Test environments use synthetic or approved de-identified data only.

- Ambiguous incident creates one task, expires, escalates, and never dispatches without approval.

- Duplicate/replayed event is idempotent; failed consumer retries then reaches DLQ with operator visibility.

- Unauthorised role cannot access evidence or approve dispatch; expired token is rejected.

- Fusion comparison reports calibration, false dispatches, verification cost and latency for Bayesian and XGBoost models.

- Load test protects operator read experience while uploads and ingestion peak; restore drill meets recovery objective.

13. Assumptions and governance

AWS is the target cloud. Human verifiers use a responsive web/PWA. Real emergency services are never automatically contacted; responder actions remain simulated or operator-approved academic demonstrations. Dataset and API use remains conditional on current license, access and institutional approval.